Universidade Federal de Lavras – UFLA/ICTIN
Engenharia de Software

Fundamentos da Medição de Software

Medição e Análise

Prof. Samira Santos da Silva

Medição na Vida Real

- A **medição** está presente em **muitos sistemas** que governam nossas vidas.

- **Exemplos:**
  - **Economia:** medições determinam preços e reajustes salariais.
  - **Medicina:** medições permitem diagnosticar doenças.
  - **Meteorologia:** medições atmosféricas possibilitam prever o clima.
  - **Tecnologia**: sistemas como radar dependem de medições para detectar objetos.

- **Sem medição, a tecnologia não consegue funcionar.**

Medição na Vida Real

- Além disso, usamos **medições no cotidiano**:
  - Comparar preços em lojas;
  - Verificar tamanho de roupas;
  - Calcular distância e tempo de viagem;
  - Medir velocidade ou combustível.

- A medição nos ajuda a **entender o mundo**, **tomar decisões** e **melhorar nossas vidas**.

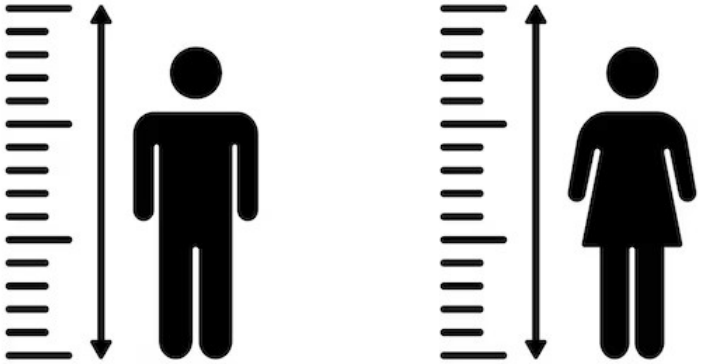
O que é Medição?

- Definição formal:
  - **Medição** é o processo de **atribuir números ou símbolos** a atributos de entidades do mundo real de acordo com regras bem definidas.

- Conceitos importantes:
  - **Entidade:** objeto ou evento do mundo real
    - Ex: uma pessoa, uma sala, uma viagem, um projeto de software.
  - **Atributo:** característica da entidade
    - Ex: altura de uma pessoa, custo de uma viagem, tempo de teste de um software, etc.

- O que medimos **não são as coisas em si**, mas os **atributos dessas coisas**.

O que é Medição?



Medição em Engenharia de Software

- **Engenharia de Software** é a aplicação de princípios de engenharia para desenvolver software de forma controlada e organizada.
- Ela envolve **atividades** como:
  - Planejamento
  - Análise de requisitos
  - Design do sistema
  - Implementação (programação)
  - Testes
  - Manutenção
- Objetivo:
  - Desenvolver software com **qualidade**, **dentro do prazo** e do **orçamento**.

Medição em Engenharia de Software

- Porém, muitos projetos ainda apresentam **problemas**:
  - **Atrasos** no cronograma;
  - **Custos maiores** que o previsto;
  - **Falhas** no software.

- Por isso, **medir o processo e o produto** de software é essencial.

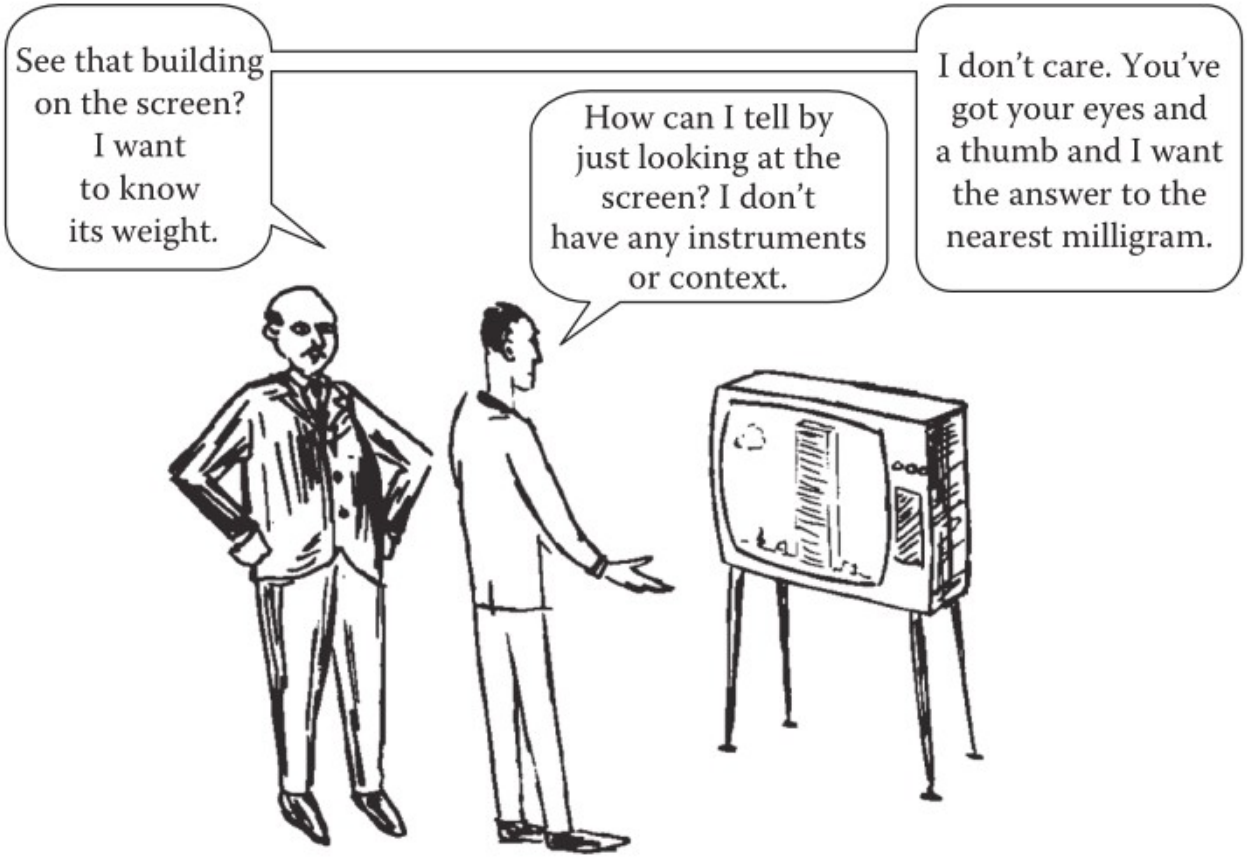
Problemas da Falta de Medição

- Na engenharia de software, muitas vezes **não medimos adequadamente**.
- Principais **problemas**:
  - **Metas não mensuráveis**:
    - Ex: Dizer que o software será “confiável” ou “fácil de usar” sem definir como medir isso.
  - **Custos mal compreendidos**:
    - Não sabemos quanto custa: requisitos, design, codificação, testes, etc.
  - **Qualidade não quantificada**:
    - Não conseguimos prever: confiabilidade, número de falhas, esforço de manutenção.
  - **Decisões baseadas em marketing ou opinião**:
    - Ex: “Ferramenta aumenta produtividade em 200%” ou “Tecnologia garante 100% de confiabilidade”.
    - Muitas dessas afirmações não são baseadas em estudos científicos.

Por que medir software?

- A medição ajuda em **três objetivos principais**:
  - **Entendimento:**
    - Compreender processos, produtos, desempenho do projeto.
  - **Controle:**
    - Monitorar o projeto para prever problemas, tomar decisões, manter prazos e custos.
  - **Melhoria:**
    - Usar dados para melhorar processos de desenvolvimento, aumentar qualidade do software e aumentar produtividade.
- Ideia central:
  - **Não é possível controlar ou melhorar um processo de software sem medi-lo.**

Por que medir software?



Escopo das Métricas de Software

- Métricas de software **não significam apenas medir código**.
  - Elas incluem várias atividades diferentes que usam medição para entender, controlar e melhorar software.

Escopo das Métricas de Software - Principais Áreas

- Estimativa de **custo** e **esforço**:
  - Uma das primeiras motivações para usar métricas foi **ajudar gerentes a prever custo e esforço** de projetos.
  - Modelos conhecidos:
    - COCOMO
    - Pontos de Função
  - Esses modelos estimam esforço com base em fatores como:
    - tamanho do software;
    - experiência da equipe;
    - reutilização de código.

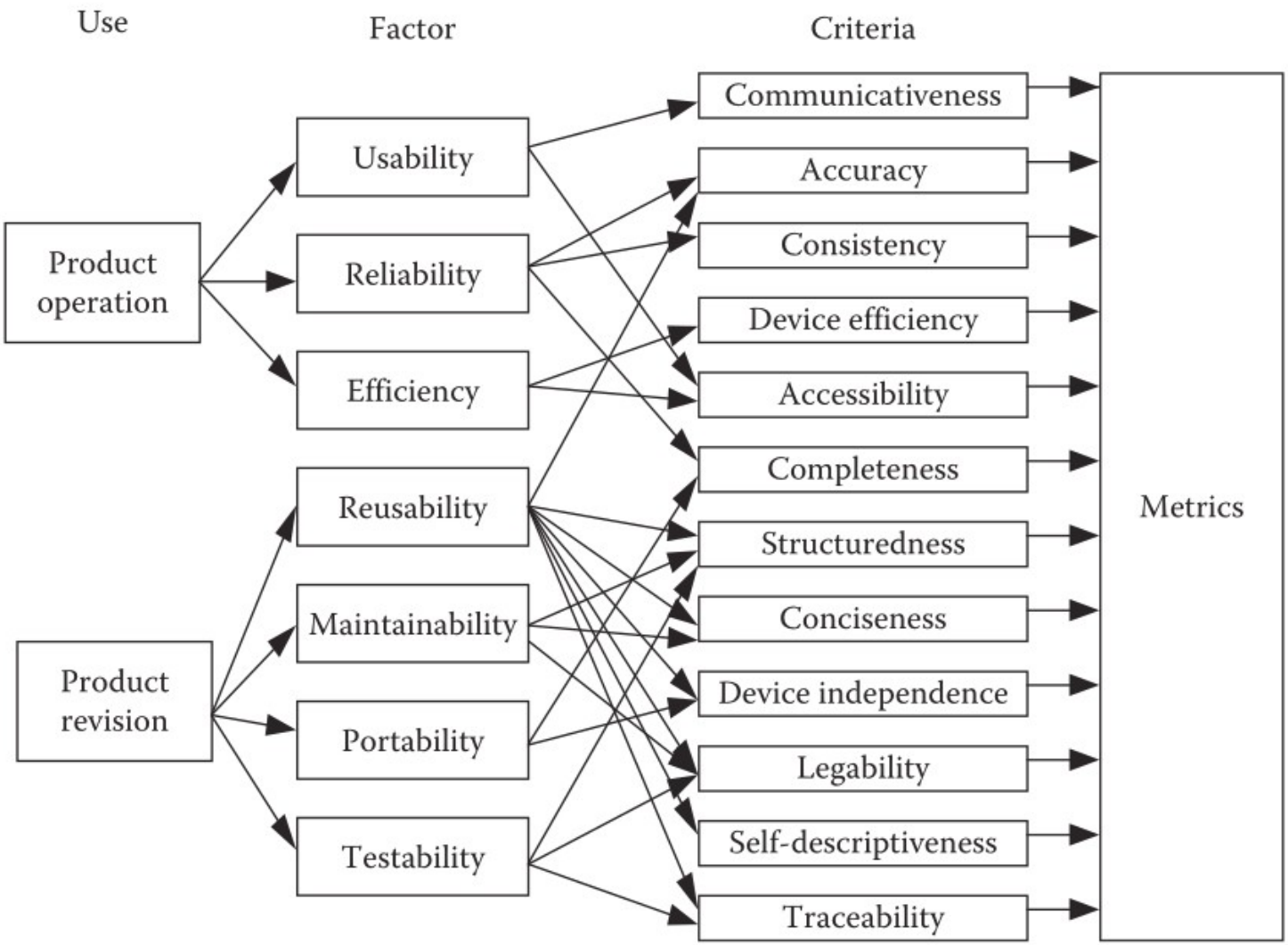
Escopo das Métricas de Software - Principais Áreas

- Coleta de dados:
  - Qualquer programa de métricas depende de **coletar dados corretamente**.
  - Isso inclui:
    - definir claramente **o que será medido;**
    - garantir **coleta consistente** entre projetos;
    - manter **integridade dos dados**.
  - Esses dados depois são usados para **análises**, **experimentos** e **estudos empíricos**.

Escopo das Métricas de Software - Principais Áreas

- Modelos de qualidade:
  - A **qualidade de software** envolve **múltiplos fatores**, como:
    - Confiabilidade;
    - Usabilidade;
    - Manutenibilidade.
  - Por isso, **modelos de qualidade** organizam esses fatores em **estruturas hierárquicas**.
  - Normalmente:
    - fatores de qualidade → nível alto
    - critérios → nível intermediário
    - métricas → nível mensurável

Escopo das Métricas de Software - Principais Áreas



Escopo das Métricas de Software - Principais Áreas

- Modelos de **confiabilidade:**
  - A confiabilidade é tão importante que existe uma área específica para isso.
  - Esses modelos ajudam a:
    - medir falhas do software;
    - prever confiabilidade durante o uso;
    - analisar comportamento do sistema ao longo do tempo.

Escopo das Métricas de Software - Principais Áreas

- Métricas de **segurança**:
  - Com o aumento da dependência de sistemas computacionais, **medir segurança tornou-se essencial**.
  - As métricas ajudam a avaliar:
    - Vulnerabilidades;
    - Ameaças;
    - Probabilidade de ataques;
    - Impacto de falhas de segurança.

Escopo das Métricas de Software - Principais Áreas

- Métricas **estruturais** e de **complexidade**:
  - Essas métricas analisam a **estrutura interna do software**, mesmo **antes de ele ser executado**.
  - Exemplos:
    - Complexidade do código;
    - Estrutura de módulos;
    - Dependências entre componentes.
  - Essas medidas ajudam a **prever problemas de qualidade**, como **baixa confiabilidade** ou **dificuldade de manutenção**.

Escopo das Métricas de Software - Principais Áreas

- Avaliação de **maturidade do processo**:
  - Métricas também são usadas para **avaliar a capacidade de uma organização** desenvolver software.
  - Exemplo:
    - CMMI (Capability Maturity Model Integration): classifica organizações em níveis de maturidade de processo.
  - Isso ajuda a entender:
    - quão estruturado é o processo de desenvolvimento;
    - quais melhorias são necessárias.

Escopo das Métricas de Software - Principais Áreas

- **Gestão** baseada em métricas:
  - Projetos de software frequentemente utilizam **indicadores e gráficos baseados em métricas** para acompanhar o progresso.
  - Isso permite:
    - monitorar andamento do projeto;
    - comparar projetos diferentes;
    - apoiar decisões de gestão.

Escopo das Métricas de Software - Principais Áreas

- Avaliação de **métodos e ferramentas**:
  - Métricas também são usadas para verificar se **novas técnicas ou ferramentas** realmente trazem **benefícios**.
  - Isso pode envolver:
    - Experimentos;
    - Estudos de caso;
    - Pesquisas empíricas.
  - O objetivo é **separar promessas de marketing de resultados reais**.

Considerações Finais

- **Medição** é fundamental na engenharia, pois permite compreender, controlar e melhorar processos e produtos.

- **Métricas de software** abrangem diversas atividades que utilizam **medição** para:
  - Estimar custo e esforço;
  - Avaliar qualidade e confiabilidade;
  - Medir complexidade e segurança;
  - Melhorar processos de desenvolvimento;
  - Apoiar a gestão de projetos.

Considerações Finais

- A **medição** deve ter **objetivos claros e bem definidos**, para que os **dados coletados** realmente **apoiem decisões**.

- A **engenharia de software** deve buscar **novas formas de medir atributos importantes**, mesmo aqueles que parecem **difíceis de quantificar**.

- O **uso de métricas** deve seguir uma **abordagem científica**, garantindo **medições confiáveis e úteis** para o desenvolvimento de software.

- Em resumo, **métricas** são **fundamentais** para tornar a **engenharia de software mais científica e controlável**.

Atividade 1

- Escolha um sistema de software que você utiliza no dia a dia.
  - Ex: WhatsApp, Instagram, Netflix, Sistema acadêmico da universidade, Aplicativo de banco, Sistema de e-commerce, etc.
- Em seguida, responda:
  1) Qual sistema você escolheu?
  2) Identifique 3 entidades do sistema.
     - Ex: sistema, módulo, funcionalidade, usuário, código, etc.
  3) Para cada entidade, identifique 2 atributos que poderiam ser medidos.
  4) Proponha 3 métricas que poderiam ser usadas para medir esse sistema.

Se eu disser que um código tem 200 linhas… isso é bom ou ruim?

Métrica ≠ Qualidade

Medição = coletar dados

Análise = interpretar dados

Atividade 2

- Considere os artefatos (texto ou código) apresentados a seguir. Calcule:
  - Número de linhas.
  - Quantidade de problemas (erros, ambiguidades, riscos).
  - Tempo gasto para análise.
  - Dar uma nota de qualidade (1 a 5).

Atividade 2

- ARTEFATO 1 — Requisitos com Problemas:
  - O sistema deve permitir o usuário fazer login com email e senha.
  - O sistema deve ser rápido.
  - O sistema deve validar os dados.
  - O sistema deve salvar os dados no banco.
  - O sistema deve ser fácil de usar.
  - O sistema deve permitir logout.

Atividade 2

- ARTEFATO 2 — Requisitos com Problemas:
  - O sistema deve permitir login com email e senha válidos.
  - O tempo de resposta do login deve ser inferior a 2 segundos.
  - O sistema deve validar se o email está no formato correto.
  - O sistema deve armazenar os dados do usuário com segurança.
  - O sistema deve permitir logout a qualquer momento.

Atividade 2

- ARTEFATO 3 — Código simples com problemas:

```python
def login(email, senha):
    if email == "admin@email.com" and senha == "123":
        return True
    else:
        return False

def calcular(a, b):
    return a / b
```

Atividade 2

- ARTEFATO 4 — Código “melhorado”:

```python
def login(email, senha):
    if not email or not senha:
        return False

    if email == "admin@email.com" and senha == "123":
        return True

    return False

def calcular(a, b):
    if b == 0:
        return None
    return a / b
```

Atividade 2

- Qual artefato teve mais problemas?

Atividade 2

- Qual artefato teve mais problemas?

- Mais linhas significa pior qualidade?

Atividade 2

- Qual artefato teve mais problemas?
- Mais linhas significa pior qualidade?
- Alguém deu nota alta mesmo com muitos problemas?

Atividade 2

- Qual artefato teve mais problemas?
- Mais linhas significa pior qualidade?
- Alguém deu nota alta mesmo com muitos problemas?
  - Dados podem ser interpretados de formas diferentes.

Atividade 3

- Use o repositório **Python Mini Projects:**
  - Uma coleção de projetos simples para aprendizado.
- Conteúdo:
  - Vários projetinhos pequenos;
  - Código simples e direto;
  - Múltiplos arquivos
- Link: https://github.com/Python-World/python-mini-projects

Atividade 3

- Escolha 3 projetos.
- Avalie:
  - Estrutura
  - Tamanho dos arquivos
  - Organização
  - Complexidade
  - Legibilidade
  - Reuso de código
  - Modularidade
  - Tratamento de erros
  - Testabilidade
  - Documentação
- Esse projeto parece de boa qualidade?

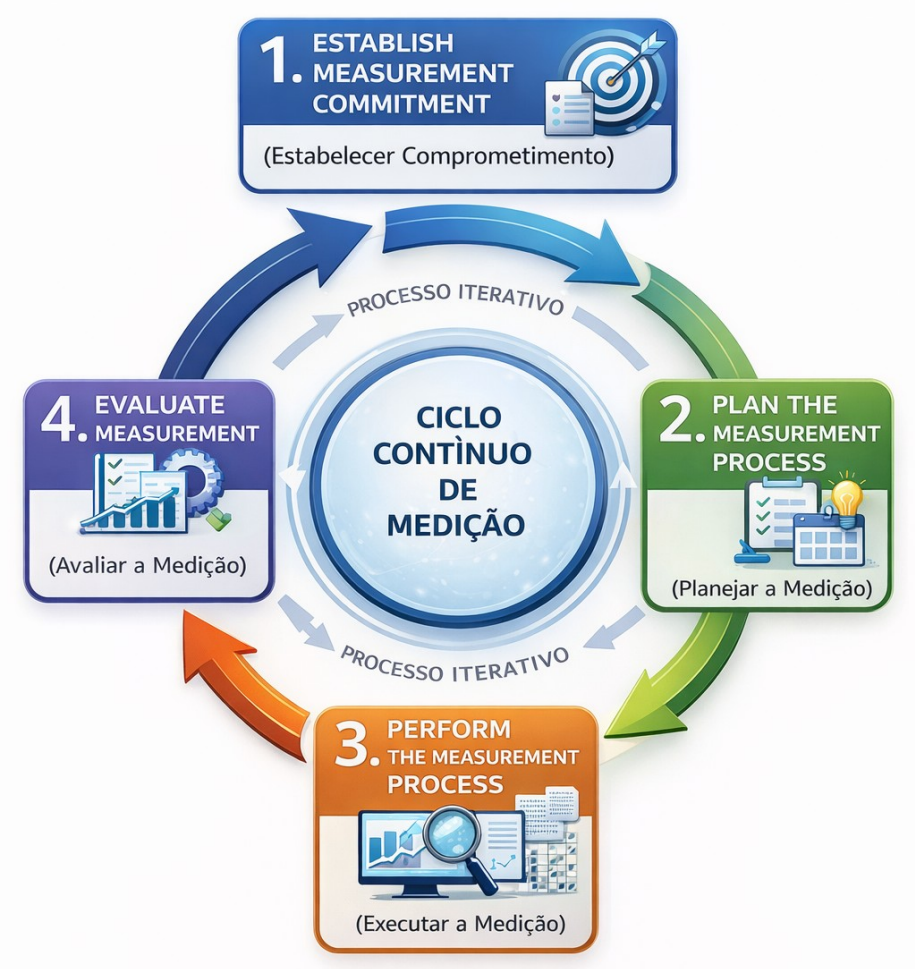
Ciclo de Medição e o Processo de Análise.

Visão Conceitual

- A medição em software não é uma atividade isolada, mas um **processo sistemático e iterativo**, cujo objetivo é:
  - Produzir **informação confiável**;
  - Reduzir incertezas;
  - Apoiar **tomada de decisão**.

Ciclo de Medição (estrutura formal)

- Segundo a **ISO 15939**, o processo de medição é organizado em um **ciclo contínuo**, composto por **4 macro atividades**:



Ciclo de Medição

- **1. Establish Measurement Commitment** (Estabelecer comprometimento):
  - Definir **por que medir**;
  - Alinhar com objetivos organizacionais;
  - Garantir apoio da gestão;
  - Sem propósito claro, métricas são inúteis.

Ciclo de Medição

- **2. Plan the Measurement Process** (Planejar a medição):
  - Aqui nasce o “coração” do ciclo - transformar objetivos em métricas mensuráveis.:
    - Definir **objetivos de medição**;
    - Identificar **entidades e atributos;**
    - Definir **métricas e indicadores**;
    - Estabelecer:
      - Métodos de coleta;
      - Frequência;
      - Responsáveis;
  - Conexão com GQM:
    - Goal → Question → Metric.

Ciclo de Medição

- **2. Plan the Measurement Process** (Planejar a medição):
  - O que é **GQM**?
    - **Goal Question Metric** é uma abordagem criada para **definir métricas de forma estruturada**, garantindo que você **não mede coisas aleatórias**.
  - Ideia central:
    - Você só define **métricas depois** de saber qual é o **objetivo**.

Ciclo de Medição

- **2. Plan the Measurement Process** (Planejar a medição):
  - **Goal** (Objetivo): O que você quer alcançar?
    - Exemplo:
      - Melhorar a qualidade do software;
      - Reduzir defeitos;
      - Aumentar produtividade.
  - **Question** (Perguntas): Que perguntas ajudam a entender se o objetivo está sendo atingido?
    - Exemplo:
      - Quantos defeitos são encontrados?
      - Em que fase eles aparecem?
      - Quanto tempo leva para corrigir?
  - **Metric** (Métricas): Quais dados você precisa coletar para responder essas perguntas?
    - Exemplo:
      - Número de defeitos;
      - Tempo de correção;
      - Taxa de defeitos por módulo.

Ciclo de Medição

- **3. Perform the Measurement Process** (Executar a medição):
  - Coleta de dados:
    - Ex: número de defeitos, tempo de execução, cobertura de testes, etc.
  - Armazenamento:
    - Dados coletados precisam ser guardados de forma organizada (ex: planilhas, banco de dados, ferramentas de monitoramento, etc).
  - Validação inicial:
    - Antes de usar os dados, você verifica se eles fazem sentido (ex: Valores faltando? Números impossíveis? Inconsistências?)
- Problemas típicos:
  - Coleta manual → erro humano;
  - Dados incompletos;
  - Dados inconsistentes;
  - Dados espalhados;
  - Falta de padronização (ex: formatos diferentes);
  - Ruído na coleta;
  - Ninguém valida → decisões erradas depois.

Ciclo de Medição

- **4. Evaluate Measurement** (Avaliar a medição):
  - Avaliar se a medição realmente foi útil e se o processo está funcionando bem.
  - Não é só olhar os dados — é avaliar o próprio processo de medição.

Ciclo de Medição

- **4. Evaluate Measurement** (Avaliar a medição):
  - Analisar resultados;
    - Você olha os dados já processados.
    - Ex:
      - Qualidade melhorou?
        Defeitos diminuíram?
      - Produtividade aumentou?
    - Aqui você começa a tirar conclusões reais.
  - Verificar se os objetivos foram atendidos;
    - Responder:
      - Conseguimos responder as perguntas do GQM?
      - Os objetivos do GQM foram alcançados?
        - Ex:
          - Objetivo: reduzir defeitos;
          - Resultado: defeitos continuam altos → ❌ objetivo não atingido;
  - Identificar melhorias no próprio processo.
    - Métricas não ajudaram → trocar métricas;
    - Coleta ruim → ajustar método;
    - Dados inconsistentes → padronizar;

Ciclo de Medição

- Visão integrada:
  - O ciclo é iterativo e adaptativo:
    - Planejar → Medir → Analisar → Avaliar → Ajustar → Repetir

Dados não têm **valor** até serem **interpretados**.

Processo de Análise

- Preparação dos Dados:
  - Antes de qualquer análise:
    - Limpeza (remoção de erros);
    - Tratamento de dados ausentes;
    - Normalização.
  - Problema clássico:
    - “Garbage in, garbage out”.

Processo de Análise

- Análise Descritiva:
  - Objetivo: entender o comportamento dos dados.
  - Inclui:
    - Média;
    - Mediana;
    - Distribuição;
    - Gráficos;
  - Exemplo:
    - Tempo médio de correção de bugs.

Processo de Análise

- Análise Interpretativa:
  - Aqui ocorre a **ligação com os objetivos**.
    - Comparar com metas;
    - Comparar com histórico;
    - Identificar tendências.
  - Exemplo:
    - Aumento de defeitos após nova versão.

Processo de Análise

- Análise Causal:
  - Pergunta central:
    - “Por que isso aconteceu?”
  - Mudança na equipe?
  - Código mais complexo?
  - Falha no processo?
  - Esse nível é essencial para melhoria real!

Processo de Análise

- Tomada de Decisão:
  - A análise só faz sentido se levar a ação:
    - Ajustes no processo;
    - Treinamento;
    - Mudança de ferramentas;
    - Revisão de arquitetura.

Processo de Análise

- Comunicação dos Resultados:
  - Relatórios;
  - Dashboards;
  - Visualizações;

  - Ponto crítico:
    - Informação não comunicada = informação perdida.

Relação entre Ciclo e Análise

- O processo de análise **não é uma etapa isolada**.

- Ele está **embutido no ciclo de medição**.

- **Relação:**
  - Planejamento define **o que analisar**;
  - Coleta fornece **dados**;
  - Análise gera **informação**;
  - Avaliação gera **decisão**.

Problemas Clássicos

- Segundo a literatura:
  - Medição sem objetivo;
  - Métricas irrelevantes;
  - Falta de análise adequada;
  - Uso incorreto de estatística;
  - Decisões baseadas em interpretação errada.

Exemplo

- **Cenário**: Sistema com aumento de defeitos.

- **Ciclo**:
  - Objetivo: reduzir defeitos;
  - Métrica: defeitos por release;
  - Coleta: sistema de issues.

- **Análise**:
  - Aumento de 30%;
  - Identificado: mudança na equipe.
- **Decisão**:
  - Treinamento + revisão de código.

Considerações Finais

- O **ciclo de medição** engloba desde a **definição** e **coleta de dados** até sua **análise e interpretação**, sendo o **processo de análise** responsável por **transformar dados em conhecimento acionáve**l para apoiar a melhoria contínua de **produtos e processos de software**.

Atividade 4

- **Cenário**: Uma equipe de desenvolvimento está enfrentando problemas de qualidade: **muitos defeitos são encontrados após a entrega**, e a equipe não entende claramente onde está o problema.
- Parte 1 – Aplicação do **GQM**:
  - Defina:
    - Goal:
      - Ex: Melhorar a qualidade do software, Reduzir defeitos em produção, etc.
    - Questions:
      - Ex: Em que fase os defeitos aparecem? Qual módulo tem mais defeitos? Quanto tempo leva para corrigir?
    - Metrics:
      - Ex: Número de defeitos por fase, Tempo médio de correção, densidade de defeitos por módulo, etc.

Atividade 4

- Parte 2 – Aplicar os 4 **macroprocessos**:
  1. Establish
  - Por que medir?
  - Quem precisa dessa informação?
  2. Plan
  - Quais métricas serão usadas?
  - Como coletar?
  - Frequência?
  - Responsáveis?
  3. Perform
  - Como os dados serão coletados na prática?
  - Onde armazenar?
  - Quais problemas podem ocorrer?
  4. Evaluate
  - Como saber se a medição foi útil?
  - O que pode ser melhorado?

Atividade 4

- Parte 3 – Problema real:
  - Sintoma: “Os dados coletados estão inconsistentes e incompletos.”
  - Identifique o problema.
  - Proponha soluções.

Dúvidas?